Laboratório de Concorrência · Aula 10

Gabriel Cesar Gravena · RA 24159150-2 · Turma ESOFT6-A
Gerado em 09/10/2026, 20:30:20 · tempo registrado no cronômetro: 25:42

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Sinconização de Cooperação é quando duas ou mais partes precisam estar alinhadas para realizar algo juntos, um exemplo sao os corredores, que passam o bastão, eles precisam alinhas sua velocida para o corredor um passar para o corredor dois. E Sinconização de competição é quando dois elementos precisam de um memso recurso, sendo ele limitado, um exemplo do dia a dia é quando varias pessoas querem usar o banheiro publico, mais possui apenas uma cabine, é um recurso que precisa ser usado por duas ou mais pessoas.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

2000000

1ª execução (n = 1.000.000)

1612655

2ª execução (n = 1.000.000)

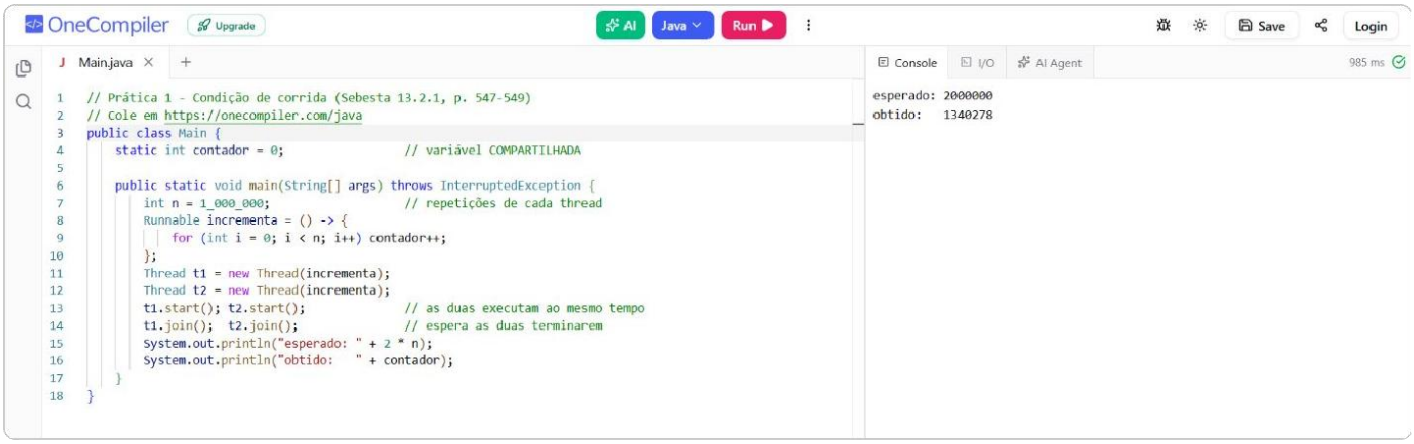
1012234

3ª execução (n = 1.000.000)

1340278

Saída do programa

não respondido







1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

nao, os valores obtidos nao batem com o resultado esperado, cada execução da um resultado diferente, e o resultado muda porque a ordem e o momento exato em que cada thread é interrompida ou executada mudam a cada rodada, alterando o número de colisões e perdas de atualização

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

A operação contador exige que aconteça três passos, ler o valor da memória, somar 1 e gravar o resultado, a perda ocorre quando a thread 1 lê o valor 10 por exemplo e antes que consiga gravar, a thread 2 lê o mesmo valor 10 ambas somam 1 chegando a 11 e gravam 11 na memória, sobrescrevendo a alteração uma da outra, e com isso, dois incrementos resultam em apenas um ganho real

1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

a execução acontece tao rápida que as threads raramente substituim uma encima da outra, fazendo com que o resultado pareça correta, esse erro é perigoso porque o código passa nos testes com baixa carga, mas falha em produção sob acesso simultâneo alto, tornando o bug muito difícil de reproduzir e corrigir

Prática 2: Todas as respostas possíveis (Figura 13.1)

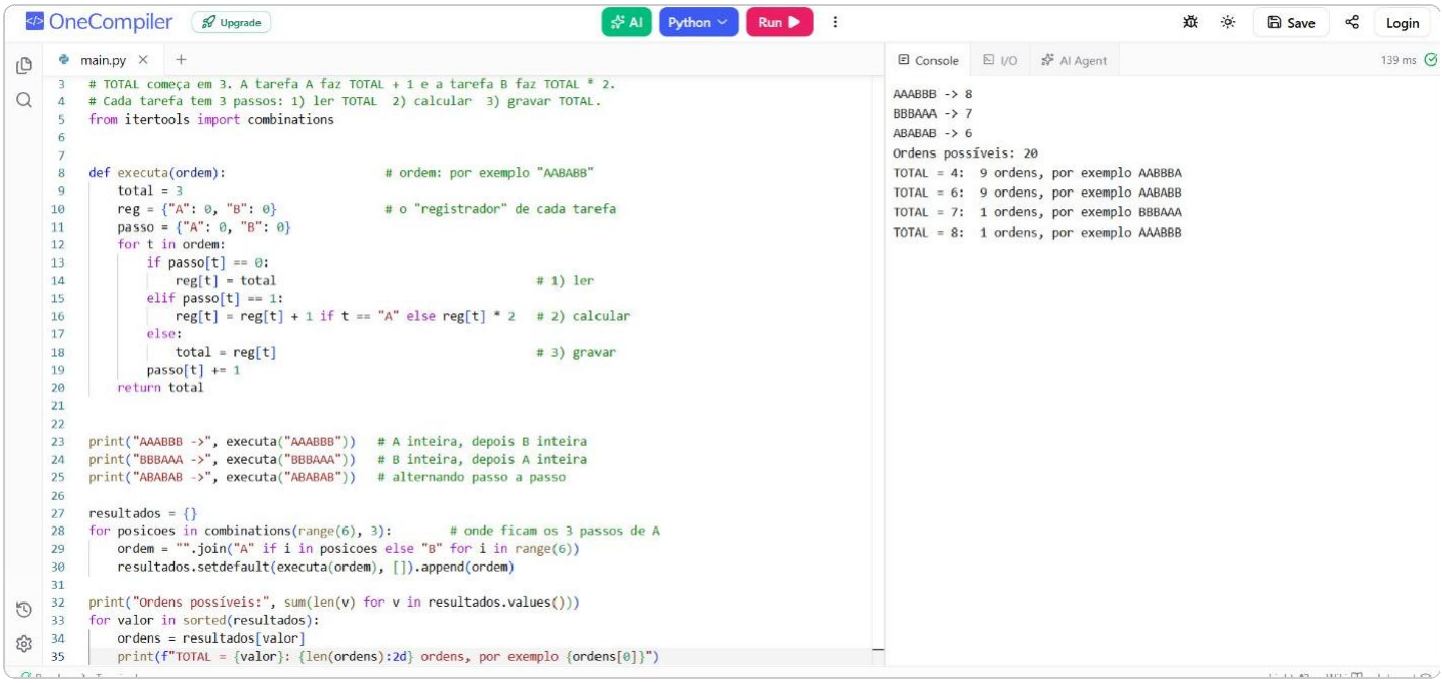
Ordens testadas no simulador

AAABBB → 8
ABABAB → 6

BBBAAA → 7
BBAABA → 4

Saída do programa

não respondido



2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

sao possiveis 20 ordens no total, e aparece 4 resultados diferentes, que são 4, 6, 7 e 8, e apenas 1 ordem AAABBB produz o resultado correto de 8 esperado

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Na ordem AABBBA, primeiro a tarefa A lê o valor de TOTAL como 3, depois em seguida A calcula a soma 3 + 1 = 4 e guarda no seu registrador, depois B lê o valor de TOTAL como 3 pois A ainda não gravou sua alteração, ai B calcula o produto 3 x 2 = 6 e grava 6 na variável TOTAL, e pra acabar, a executa seu passo de gravação e escreve o valor 4 na variável TOTAL, e com relação à operação de B, ela foi totalmente perdida e sobrescrita, como B leu o valor inicial antes de A salvar e A gravou por último, o cálculo feito por B foi apagado e substituído pelo resultado de A

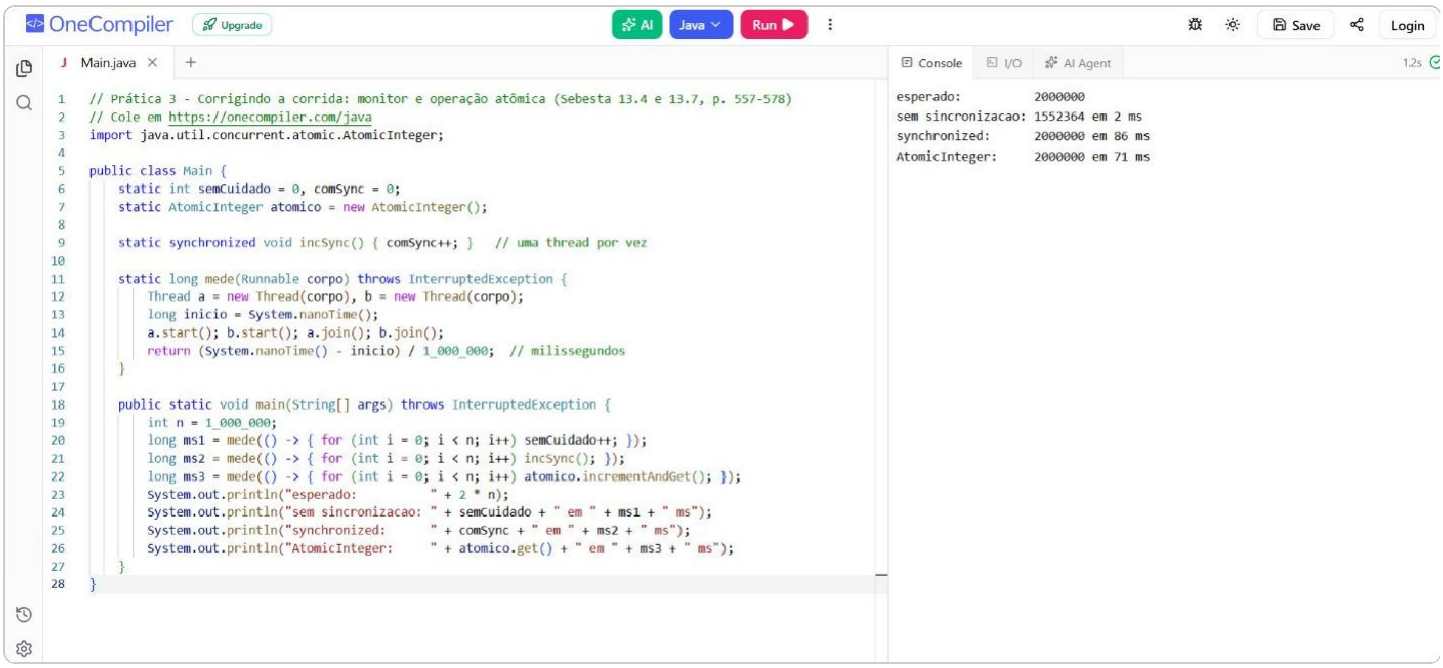
Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

A mais rapida que eu acho vai ser a Operação atômica, e acredito que nenhuma das três sera incorreta

Saída do programa

não respondido



3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

As versões com Monitor e Operação atômica deram o valor correto, enquanto a versão sem sincronização deu um valor incorreto. Nos tempos de execução, a versão sem sincronização levou 2ms, a com Operação atômica levou 71ms e a com Monitor levou 86ms, as versões corretas são mais lentas porque exigem mecanismos de controlo de concorrência, como bloqueios por exemplo, exclusão mútua etc, operacional para garantir a integridade dos dados, o que gera sobrecustos e torna a execução mais demorada do que o acesso sem sincronização

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não resolve, porque usar uma Operação atômica dividual apenas garante que cada operação isolada seja realizada como um passo só, mas não garante a exclusão mútua ou a integridade das operações. Se o sistema falhar entre o débito numa conta e o crédito na outra, o dinheiro perde-se, gerando inconsistência, e para resolver isto, seria necessário um mecanismo como um Monitor ou um Semáforo a envolver ambas as contas, de modo a garantir que a transferência completa ocorra como uma única unidade indivisível

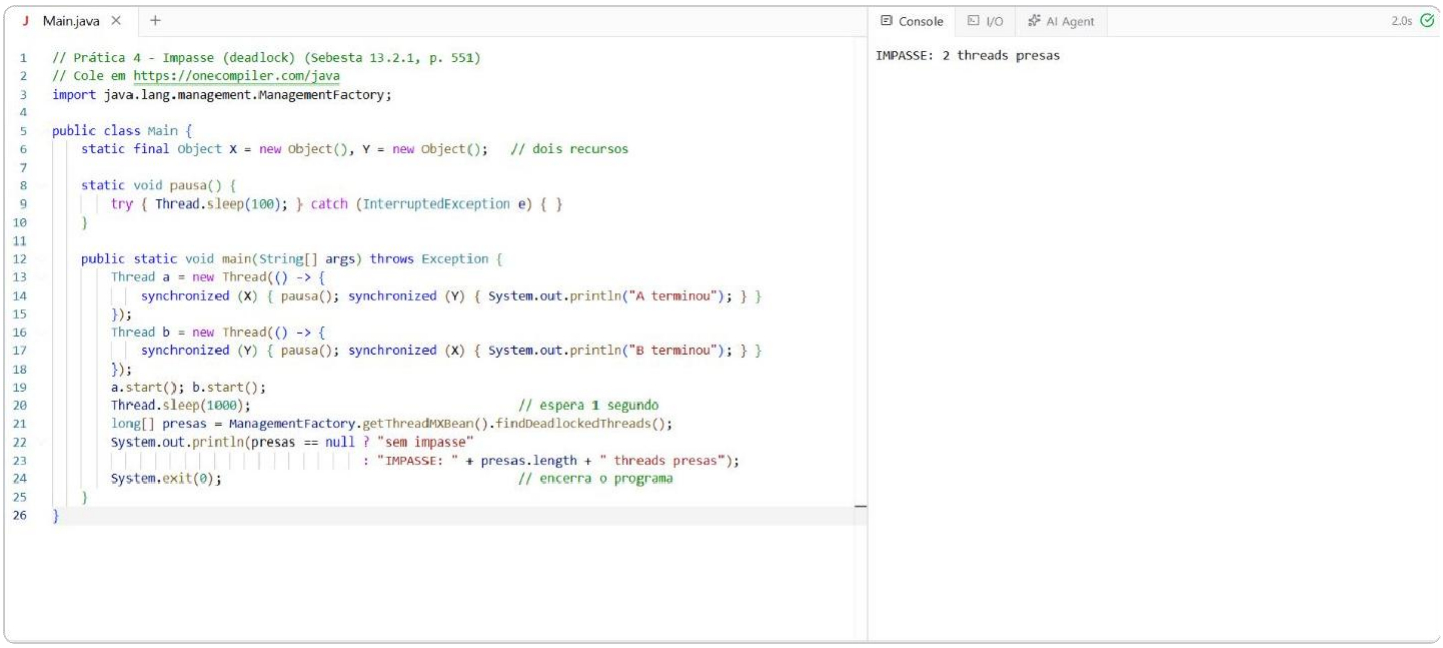
Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

As mensagens não vão aparecer, como a primeira tarefa pega no recurso X e espera pelo Y, e a segunda pega no Y e espera pelo X, ambas ficam presas esperando uma da outra para sempre, ai o programa simplesmente para e nenhuma das duas consegue concluir a execução

Saídas (antes e depois da correção)

não respondido



4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

As tarefas ficaram presas porque uma depende da outra para continuar, a primeira tarefa tinha o recurso X e esperava pelo recurso Y, ao mesmo tempo, a segunda tarefa tinha o recurso Y e esperava pelo recurso X, ai como nenhuma abre mão do que tem, ambas ficam bloqueadas para sempre

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Pegar os recursos sempre na mesma ordem elimina o impasse porque impede a espera circular, se ambas tentarem pegar primeiro o recurso X, a segunda tarefa fica a aguardar antes de pegar o recurso Y, garantindo que a primeira termine o seu trabalho e liberte tudo, um exemplo real disso é uma transferência bancária entre duas contas, onde o sistema deve sempre bloquear primeiro a conta com o menor código de identificação e depois a outra para evitar que duas transferências simultâneas fiquem presas

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Com 3 trabalhadores, espera que leve cerca de 200 ms no total, pois cada um processará 2 tarefas em paralelo, com 1 trabalhador, o tempo total será de 600 ms, já que ele processará todas as 6 tarefas sequencialmente e com 6 trabalhadores, levará aproximadamente 100 ms, pois haverá um trabalhador dedicado a cada uma das 6 tarefas ao mesmo tempo

Tempo com 1 trabalhador

levou 600ms

Tempo com 3 trabalhadores

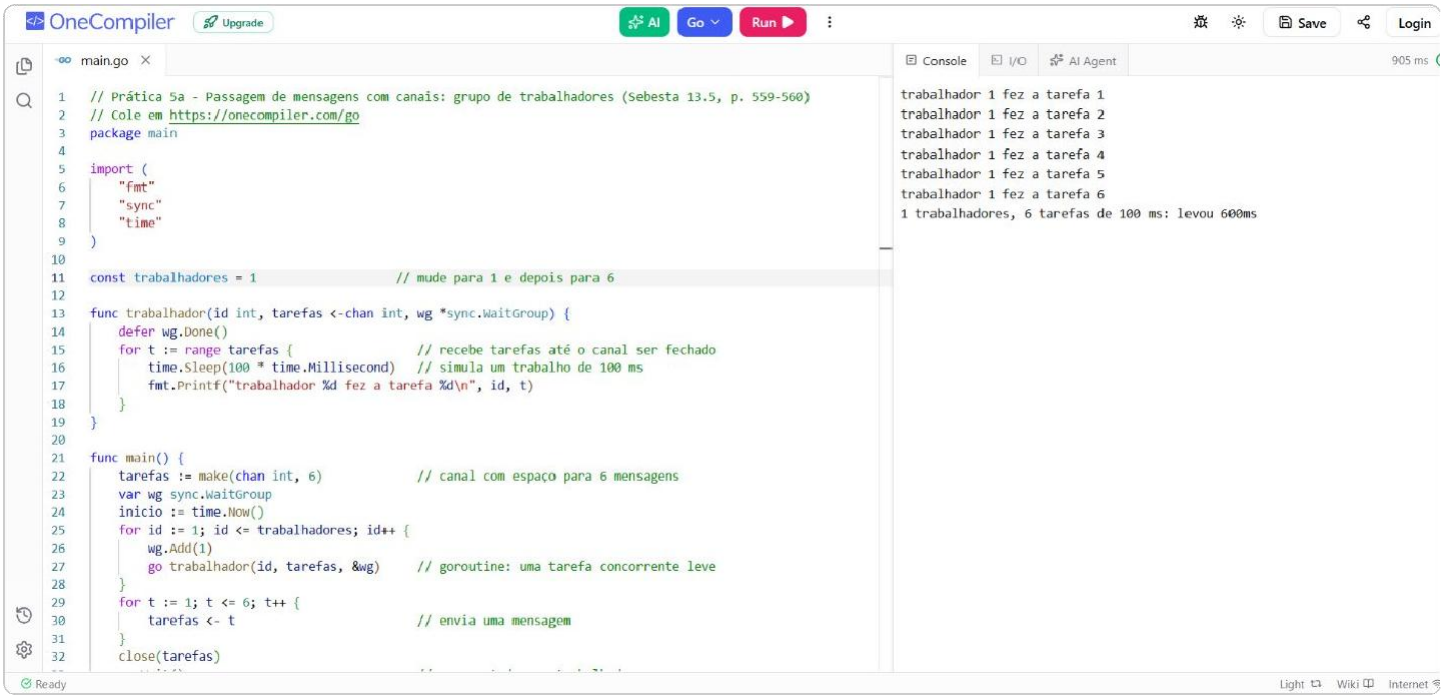
levou 200ms

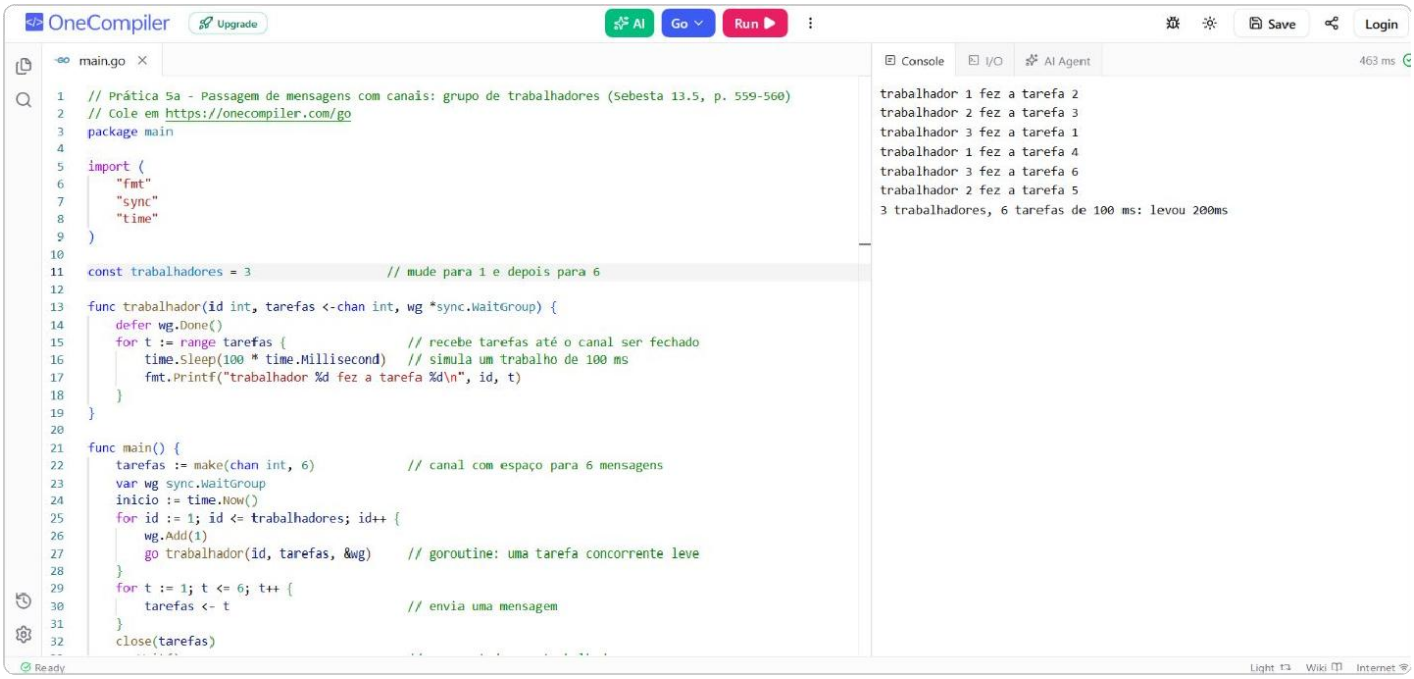
Tempo com 6 trabalhadores

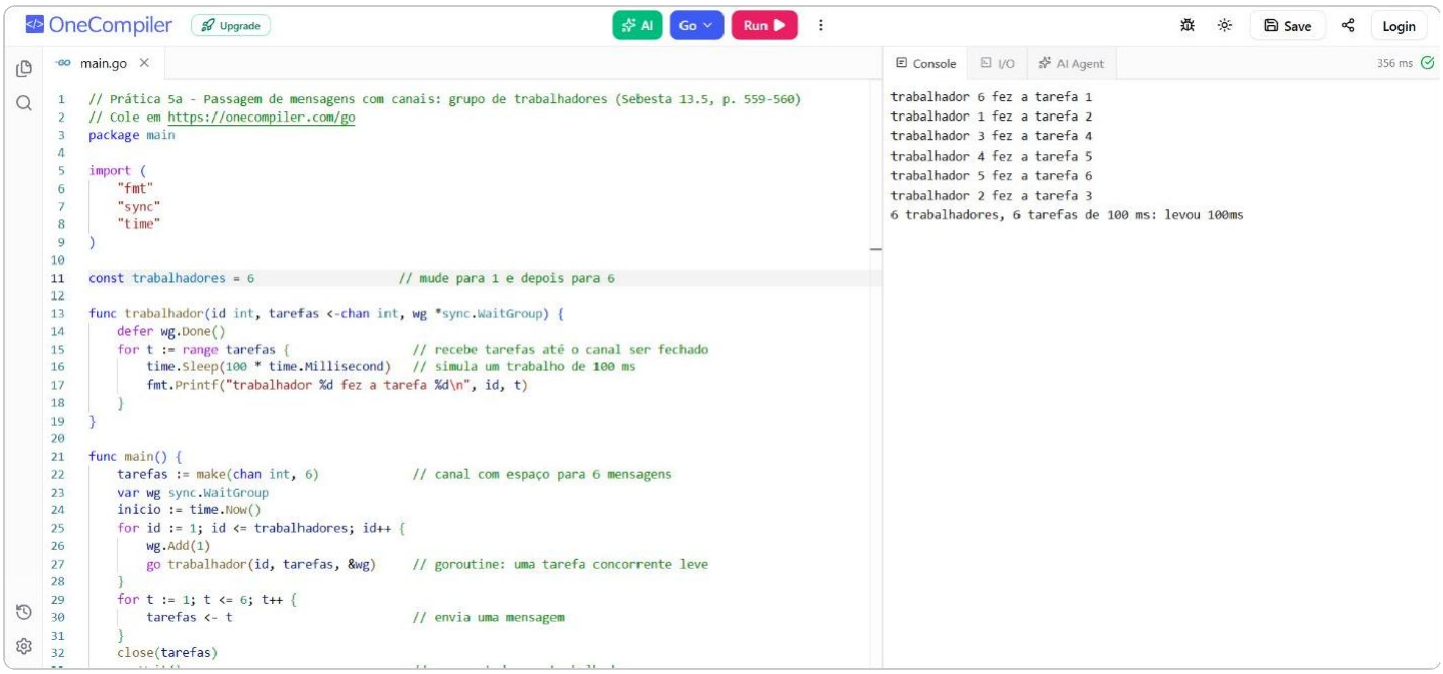
levou 100ms

Saída do programa

não respondido


OneCompiler
Upgrade
AI
Go
Run
Save
Login
main.go
Console
I/O
AI Agent
905 ms
// Prática 5a - Passagem de mensagens com canais: grupo de trabalhadores (Sebesta 13.5, p. 559-560)
// Cole em https://onecompiler.com/go
package main
import (
"fmt"
"sync"
"time"
)
const trabalhadores = 1 // mude para 1 e depois para 6
func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
defer wg.Done()
for t := range tarefas { // recebe tarefas até o canal ser fechado
time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
}
}
func main() {
tarefas := make(chan int, 6) // canal com espaço para 6 mensagens
var wg sync.WaitGroup
inicio := time.Now()
for id := 1; id <= trabalhadores; id++ {
wg.Add(1)
go trabalhador(id, tarefas, &wg) // goroutine: uma tarefa concorrente leve
}
for t := 1; t <= 6; t++ {
tarefas <- t // envia uma mensagem
}
close(tarefas)
trabalhador 1 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 1 fez a tarefa 3
trabalhador 1 fez a tarefa 4
trabalhador 1 fez a tarefa 5
trabalhador 1 fez a tarefa 6
1 trabalhadores, 6 tarefas de 100 ms: levou 600ms
Ready
Light
Wiki
Internet


OneCompiler
Upgrade
AI
Go
Run
Save
Login
main.go
Console
I/O
AI Agent
463 ms
// Prática 5a - Passagem de mensagens com canais: grupo de trabalhadores (Sebesta 13.5, p. 559-560)
// Cole em https://onecompiler.com/go
package main
import (
"fmt"
"sync"
"time"
)
const trabalhadores = 3 // mude para 1 e depois para 6
func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
defer wg.Done()
for t := range tarefas { // recebe tarefas até o canal ser fechado
time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
}
}
func main() {
tarefas := make(chan int, 6) // canal com espaço para 6 mensagens
var wg sync.WaitGroup
inicio := time.Now()
for id := 1; id <= trabalhadores; id++ {
wg.Add(1)
go trabalhador(id, tarefas, &wg) // goroutine: uma tarefa concorrente leve
}
for t := 1; t <= 6; t++ {
tarefas <- t // envia uma mensagem
}
close(tarefas)
trabalhador 1 fez a tarefa 2
trabalhador 2 fez a tarefa 3
trabalhador 3 fez a tarefa 1
trabalhador 1 fez a tarefa 4
trabalhador 3 fez a tarefa 6
trabalhador 2 fez a tarefa 5
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms
Ready
Light
Wiki
Internet



5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Com 1 trabalhador, o tempo total é de 600ms porque o único trabalhador executa as 6 tarefas de forma sequencial, com 3 trabalhadores, o tempo desce para 200 ms, pois as 6 tarefas são distribuídas e executadas em paralelo, cabendo duas tarefas a cada trabalhador , com 6 trabalhadores, o tempo reduz-se para 100 ms, uma vez que existe capacidade para processar todas as 6 tarefas em simultâneo, com um trabalhador dedicado a cada uma delas

Saídas (antes e depois da troca)

não respondido





5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

O primeiro programa trava porque cria um canal sem espaço, onde quem envia fica bloqueado à espera que alguém esteja pronto para receber a mensagem no mesmo instante, como não há outra para recolher o valor, o programa gera um erro de impasse, o canal com espaço para 1 mensagem resolve o problema porque permite que

o valor seja guardado temporariamente no canal, permitindo que a linha de envio conclua sem precisar que o recetor esteja a retirar o dado de imedia

Prática 6: O compilador como guarda

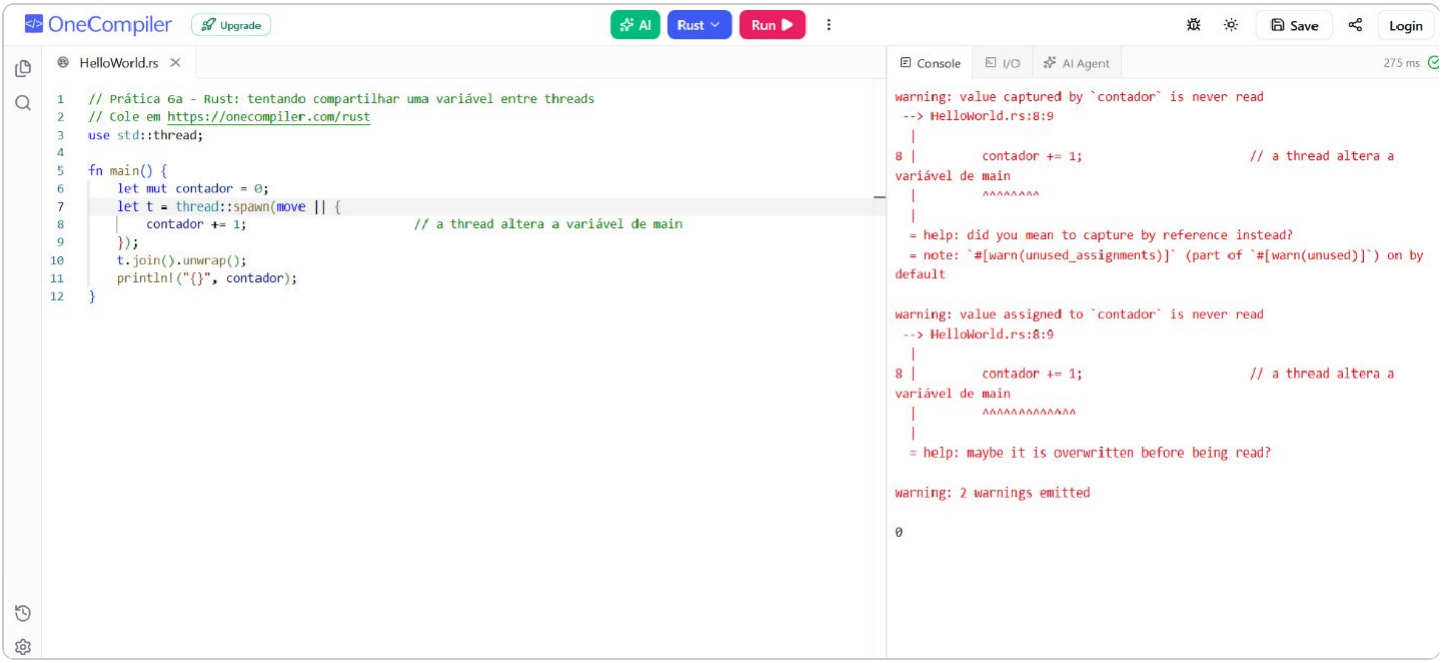
Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

Acredito que vai gerar um erro

Saídas (o erro e o resultado com move)

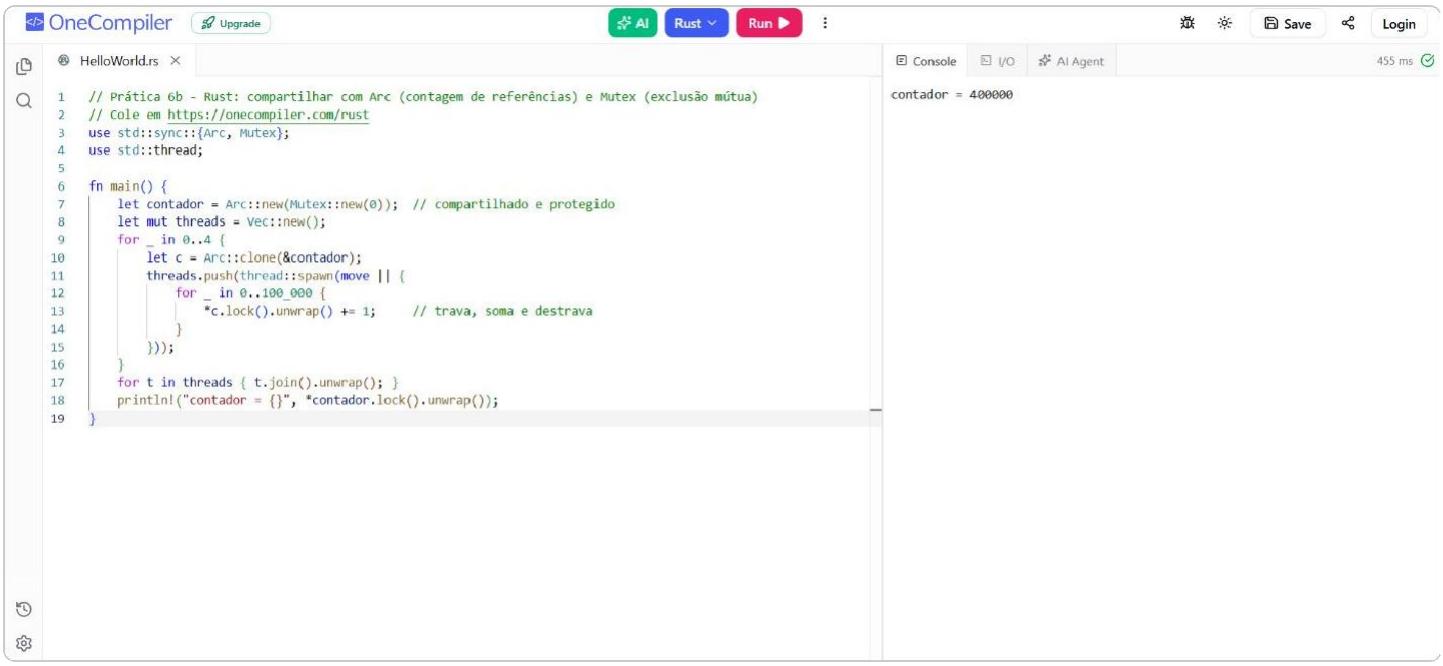
não respondido





Saída do programa

não respondido



6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

O uso do move faz com que cada thread leve uma cópia própria ou transfira a propriedade da variável, fazendo com que as alterações ocorram em variáveis isoladas ou cópias separadas do contador original na função principal, resultando na impressão do valor inicial zero ao final

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java, o erro de concorrência não impede a compilação e só aparece durante a execução, podendo passar despercebido. Em Rust, o compilador bloqueia problemas de partilha insegura logo na compilação, a vantagem do Rust é a maior segurança contra bugs, enquanto o custo é a exigência de regras mais rígidas e difíceis de aprender para quem programa.

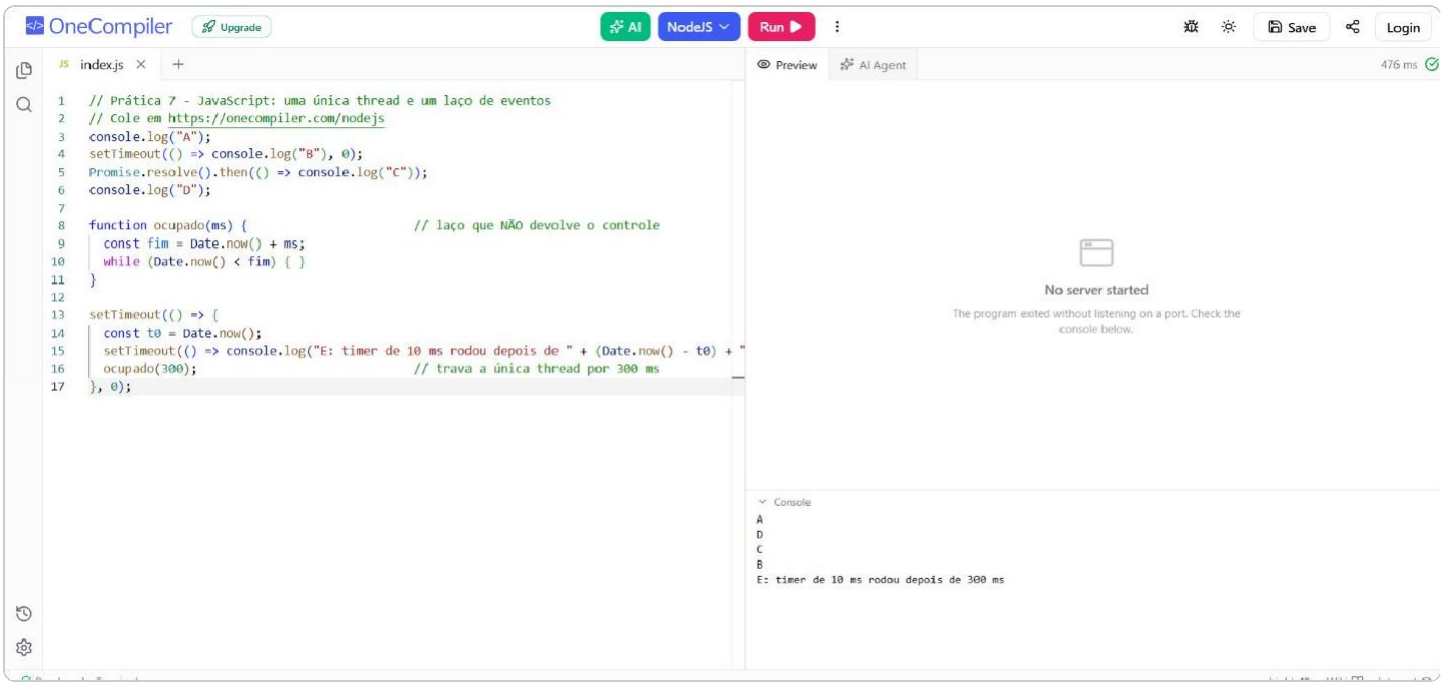
Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

a d c b e

Saída do programa

não respondido



7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

As letras saíram na ordem A, D, C, B, E. O B saiu depois de D e C porque o setTimeout vai para a fila de tarefas assíncronas , enquanto as chamadas síncronas rodam imediatamente e as micro-tarefas de Promessas têm prioridade sobre as tarefas de temporizadores

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O timer de 10 ms rodou muito depois porque a única thread do node ficou travada por 300 ms num laço de repetição, bloqueando a execução de qualquer outra tarefa da fila, para um servidor que atende muitos usuários, isso significa que qualquer cálculo pesado ou bloqueante paralisa o servidor inteiro, fazendo com que todos os outros pedidos fiquem congelados à espera que a thread volte a ficar livre

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

O problema que pode acontecer é uma condição onde ambas as transações verificam ao mesmo tempo que há um ingresso disponível, permitindo que os dois usuários comprem o último lugar e gerando uma venda dupla, para evitar isso, eu utilizaria mecanismos de sincronização ou exclusão mútua, garantindo que a verificação de disponibilidade e a efetivação da compra ocorram de forma indivisível e que apenas um dos pedidos seja processado por vez

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais surpreendeu nesta atividade foi perceber como problemas de concorrência invisíveis e difíceis de reproduzir e identificar e podem corromper dados silenciosamente em sistemas reais se não forem prevenidos desde a conceção